

DATA ANALYSIS AND DATA SCIENCE WITH PYTHON

TASK – 2

Exploratory Data Analysis (EDA)

Project 1: General EDA

Step 1: Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Step 2: Load the Dataset

```
df = pd.read_csv("C:/Users/Prathamesh/Desktop/purwa placement/internship/global_superstore_sample.csv")
df.head()
```

	Order_ID	Region	Category	Sales	Profit
0	1	Central	Technology	5946.48	970.88
1	2	South	Technology	4854.34	1738.09
2	3	East	Furniture	3306.41	1275.21
3	4	Central	Furniture	1970.31	511.55
4	5	Central	Office Supplies	4106.97	NaN

Step 3: Understand the Data

```
[5]: df.shape
      df.info()
      df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 205 entries, 0 to 204
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Order_ID    205 non-null   int64
1   Region      205 non-null   object
2   Category    205 non-null   object
3   Sales       195 non-null   float64
4   Profit      197 non-null   float64
dtypes: float64(2), int64(1), object(2)
memory usage: 8.1+ KB
```

	Order_ID	Sales	Profit
count	205.000000	195.000000	197.000000
mean	99.882927	5134.989744	795.329492
std	58.693754	1903.015760	508.091493
min	1.000000	-1482.530000	-350.960000
25%	49.000000	3777.085000	425.760000
50%	100.000000	5163.750000	783.650000
75%	151.000000	6313.520000	1122.690000
max	200.000000	12705.460000	2339.440000

Step 4: Data Cleaning

```
6]: df.isnull().sum()
```

```
6]: Order_ID      0
     Region      0
     Category     0
     Sales       10
     Profit       8
     dtype: int64
```

Step 5: Handle Missing Values

Fill missing values with **mean**:

```

: df["Sales"].fillna(df["Sales"].mean())
  df["Profit"].fillna(df["Profit"].mean())

: 0      970.880000
  1     1738.090000
  2     1275.210000
  3      511.550000
  4      795.329492
   ...
 200    1045.960000
 201      295.960000
 202    1389.720000
 203    1476.940000
 204    1383.890000
  Name: Profit, Length: 205, dtype: float64

```

Step 6: Remove Duplicates

```

df.duplicated().sum()
df.drop_duplicates(inplace=True)

```

Step 7: 4.3 Outlier Detection & Removal (IQR Method)

```

Q1 = df["Sales"].quantile(0.25)
Q3 = df["Sales"].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

df = df[(df["Sales"] >= lower_bound) & (df["Sales"] <= upper_bound)]

```

Step 8: Statistical Analysis

Mean, Median, Standard Deviation, Variance

```
print("Mean:\n", df[["Sales", "Profit"]].mean())
print("Median:\n", df[["Sales", "Profit"]].median())
print("Standard Deviation:\n", df[["Sales", "Profit"]].std())
print("Variance:\n", df[["Sales", "Profit"]].var())
```

Mean:

```
Sales      5112.842714
Profit      787.589474
dtype: float64
```

Median:

```
Sales      5134.989744
Profit      795.329492
dtype: float64
```

Standard Deviation:

```
Sales      1739.786542
Profit      498.335702
dtype: float64
```

Variance:

```
Sales      3.026857e+06
Profit      2.483385e+05
dtype: float64
```

Correlation

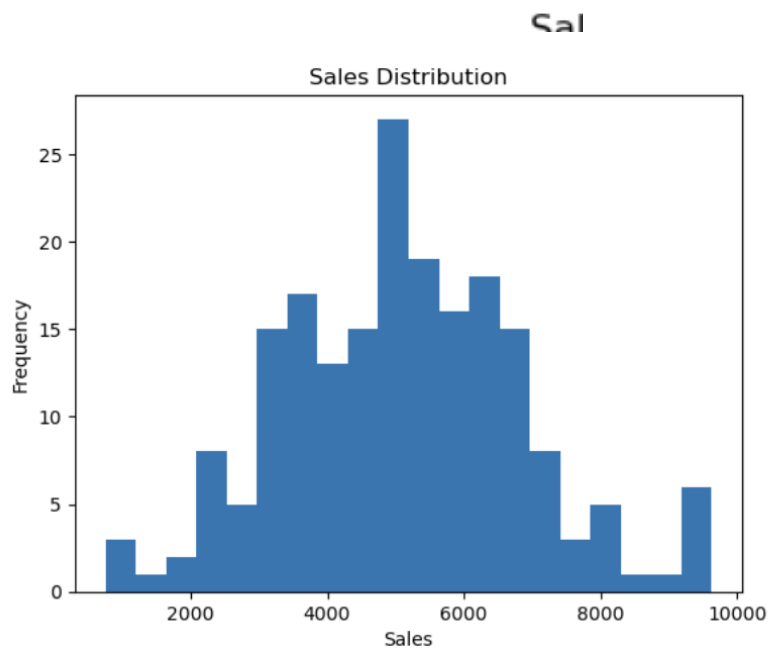
```
: corr = df[["Sales", "Profit"]].corr()
print(corr)
```

```
          Sales  Profit
Sales  1.000000 -0.067001
Profit -0.067001  1.000000
```

Step 9: Data Visualization

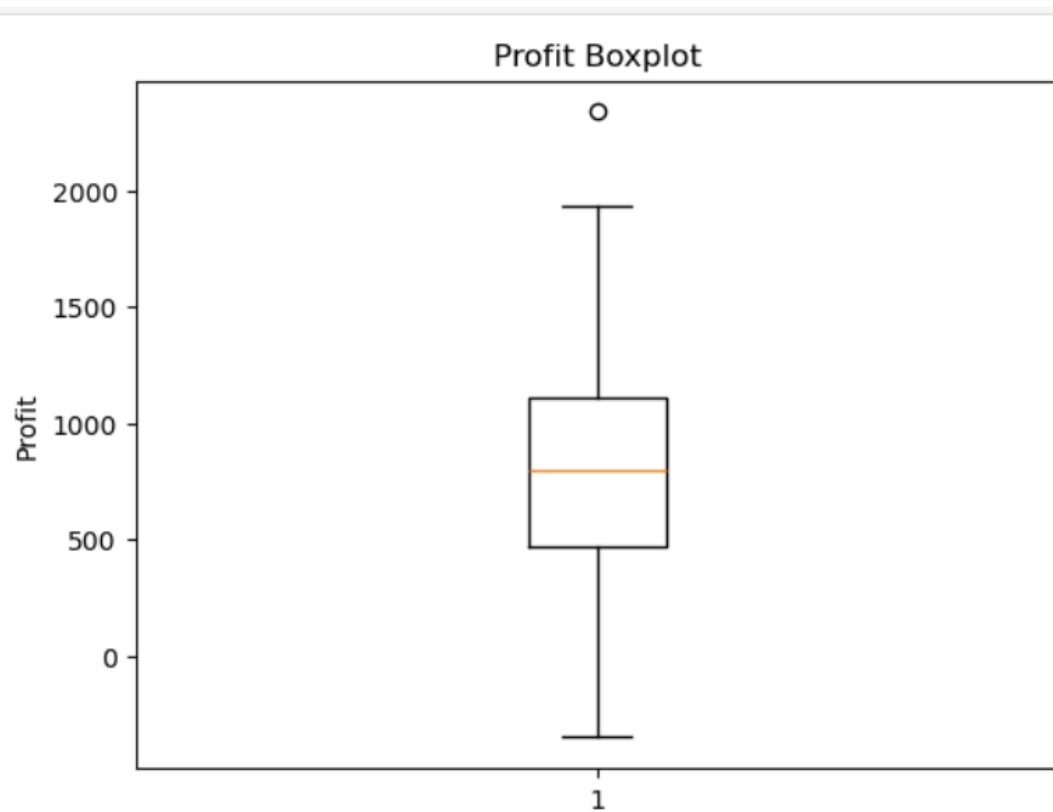
a) Histogram – Sales Distribution

```
plt.figure()
plt.hist(df["Sales"], bins=20)
plt.title("Sales Distribution")
plt.xlabel("Sales")
plt.ylabel("Frequency")
plt.show()
```



b) Boxplot – Profit (Outliers)

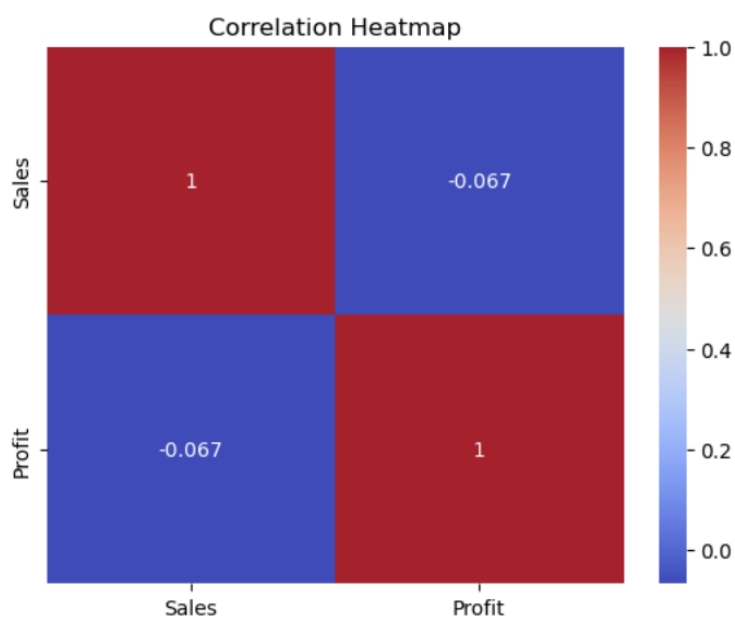
```
plt.figure()
plt.boxplot(df["Profit"])
plt.title("Profit Boxplot")
plt.ylabel("Profit")
plt.show()
```



c)Heatmap

```
import seaborn as sns
corr = df[["Sales", "Profit"]].corr()

plt.figure()
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```



Project 2: Sales Performance Analysis

Step 1: Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

Step 2: Load and Explore the Dataset

```
df = pd.read_csv("C:/Users/Prathamesh/Desktop/purwa placement/internship/sales_data.csv")
df.head()
df.shape
df.info()
df.isnull().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 305 entries, 0 to 304
```

```
Data columns (total 7 columns):
```

#	Column	Non-Null Count	Dtype
0	Product	305 non-null	object
1	Region	305 non-null	object
2	Category	305 non-null	object
3	Sales	295 non-null	float64
4	Profit	297 non-null	float64
5	Discount	299 non-null	float64
6	Date	305 non-null	object

```
dtypes: float64(3), object(4)
```

```
memory usage: 16.8+ KB
```

```
Product      0
Region       0
Category     0
Sales        10
Profit        8
Discount      6
Date          0
dtype: int64
```

Step 3: Data Cleaning

a) Remove Duplicates

```
df.drop_duplicates(inplace=True)
```

b) Handle Missing Values

```
: df["Sales"].fillna(df["Sales"].mean())
df["Profit"].fillna(df["Profit"].mean())
df["Discount"].fillna(df["Discount"].mean())

: 0      0.390000
  1      0.330000
  2      0.250000
  3      0.280000
  4      0.360000
  ...
295    0.253741
296    0.480000
297    0.090000
298    0.350000
299    0.100000
Name: Discount, Length: 300, dtype: float64
```

c) Convert Date Column to Datetime

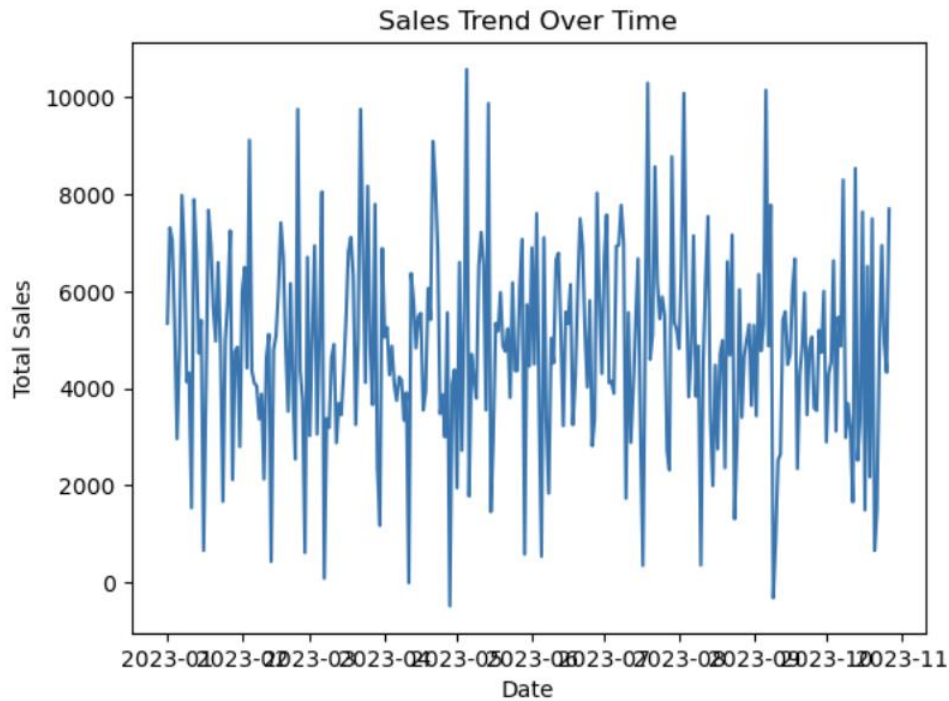
```
df["Date"] = pd.to_datetime(df["Date"])
```

Step 4: Exploratory Data Analysis (EDA)

a) Time Series Analysis – Sales Trend Over Time

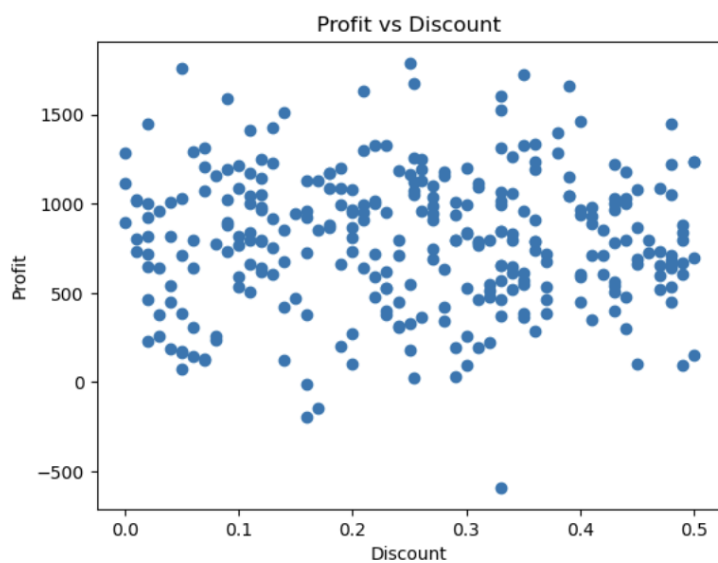
```
sales_over_time = df.groupby("Date")["Sales"].sum()

plt.figure()
plt.plot(sales_over_time)
plt.title("Sales Trend Over Time")
plt.xlabel("Date")
plt.ylabel("Total Sales")
plt.show()
```



b) Scatter Plot – Profit vs Discount

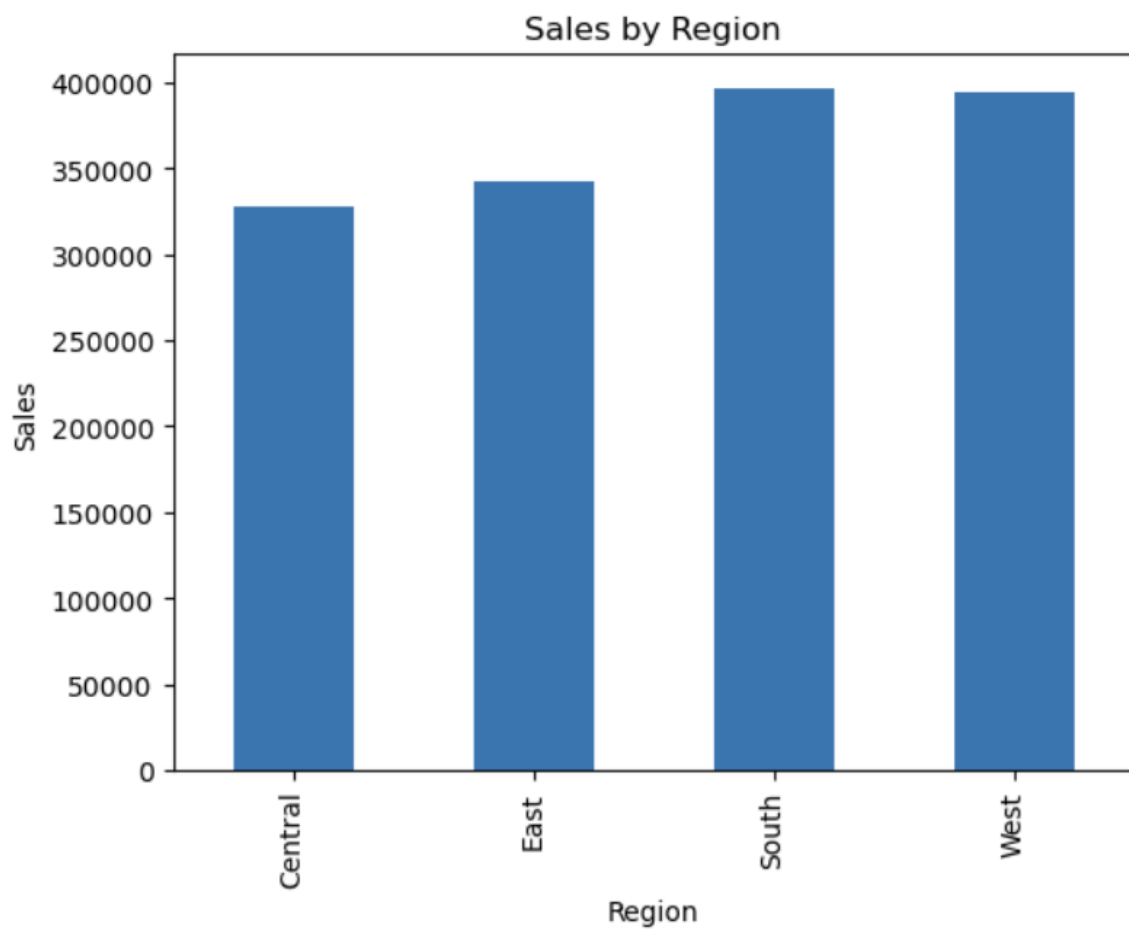
```
plt.figure()
plt.scatter(df["Discount"], df["Profit"])
plt.title("Profit vs Discount")
plt.xlabel("Discount")
plt.ylabel("Profit")
plt.show()
```



c) Sales by Region (Bar Chart)

```
region_sales = df.groupby("Region")["Sales"].sum()

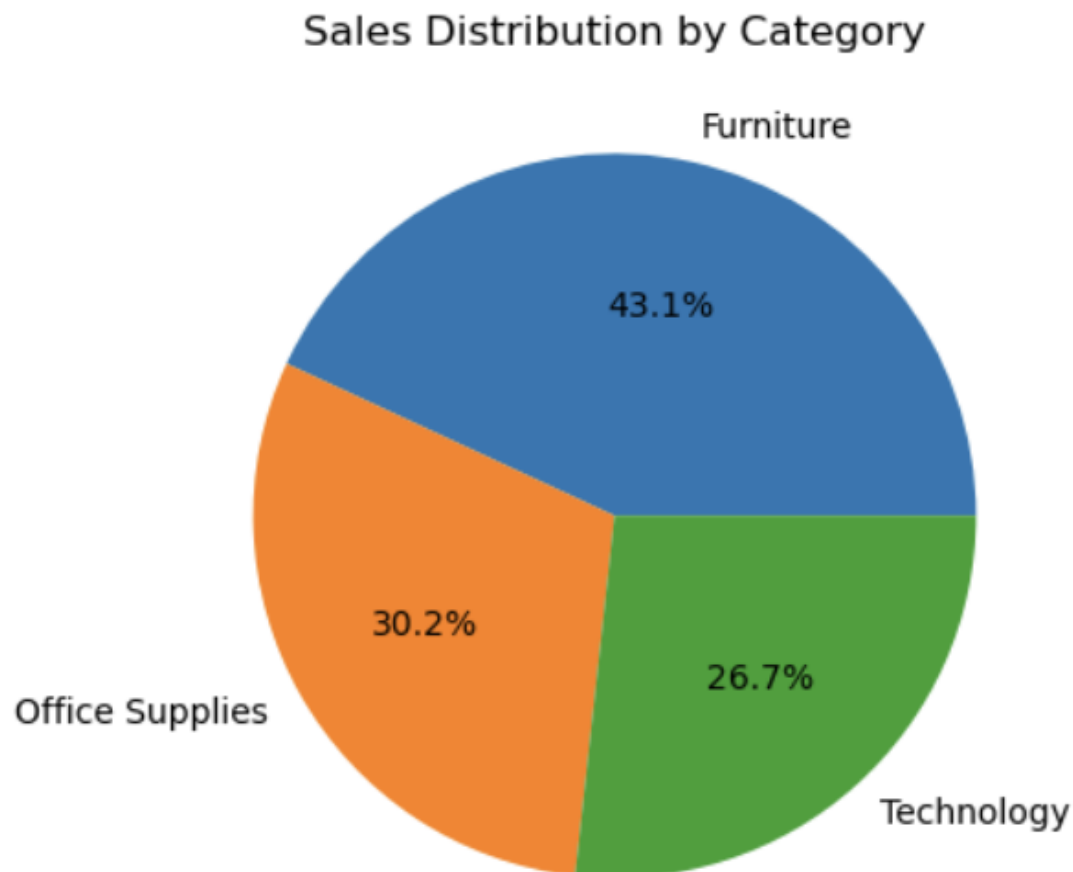
plt.figure()
region_sales.plot(kind="bar")
plt.title("Sales by Region")
plt.xlabel("Region")
plt.ylabel("Sales")
plt.show()
```



d) Sales by Category (Pie Chart)

```
category_sales = df.groupby("Category")["Sales"].sum()

plt.figure()
plt.pie(category_sales, labels=category_sales.index, autopct='%1.1f%%')
plt.title("Sales Distribution by Category")
plt.show()
```



Step 5: Predictive Modeling (Linear Regression)

a) Features & Target

```
: X = df[["Profit", "Discount"]]
  y = df["Sales"]
```

b) Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)
```

c) Train Linear Regression Model

```
: model = LinearRegression()
  model.fit(X_train, y_train)
```

```
: ▼ LinearRegression ⓘ ?
  LinearRegression()
```

d) Model Evaluation

```
y_pred = model.predict(X_test)

print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

Mean Squared Error: 3305405.3554488197

R2 Score: -0.016592247648882452

e) Model Coefficients

```
: coeff_df = pd.DataFrame(model.coef_, X.columns, columns=["Coefficient"])
  print(coeff_df)
```

	Coefficient
Profit	-0.325034
Discount	-1980.810299
